

# Password-Hardened Encryption Revisited

Ruben Baecker<sup>1</sup>, Paul Gerhart<sup>2</sup>, and Dominique Schröder<sup>2,1</sup>

<sup>1</sup> Friedrich-Alexander-Universität Erlangen-Nürnberg, Erlangen, Germany

`ruben.baecker@fau.de`

<sup>2</sup> TU Wien, Vienna, Austria

`{paul.gerhart, dominique.schroeder}@tuwien.ac.at`

**Abstract.** Passwords remain the dominant form of authentication on the Internet. The rise of single sign-on (SSO) services has centralized password storage, increasing the devastating impact of potential attacks and underscoring the need for secure storage mechanisms. A decade ago, Facebook introduced a novel approach to password security, later formalized in Pythia by Everspaugh et al. (USENIX’15), which proposed the concept of password hardening. The primary motivation behind these advances is to achieve provable security against offline brute-force attacks. This work initiated significant follow-on research (CCS’16, USENIX’17), including Password-Hardened Encryption (PHE) (USENIX’18, CCS’20), which was introduced shortly thereafter. Virgil Security commercializes PHE as a software-as-a-service solution and integrates it into its messenger platform to enhance security.

In this paper, we revisit PHE and provide both negative and positive contributions. First, we identify a critical weakness in the original design (USENIX’18) and present a practical cryptographic attack that enables offline brute-force attacks – the very threat PHE was designed to mitigate. This weakness stems from a flawed security model that fails to account for real-world attack scenarios and the interaction of security properties with key rotation, a mechanism designed to enhance security by periodically updating keys. Our analysis shows how the independent treatment of security properties in the original model leaves PHE vulnerable. We demonstrate the feasibility of the attack by extracting passwords in seconds that were secured by the commercialized but open-source PHE provided by Virgil Security.

On the positive side, we propose a novel, highly efficient construction that addresses these shortcomings, resulting in the first practical PHE scheme that achieves security in a realistic setting. We introduce a refined security model that accurately captures the challenges of practical deployments, and prove that our construction meets these requirements. Finally, we provide a comprehensive evaluation of the proposed scheme, demonstrating its robustness and performance.

## 1 Introduction

Passwords remain popular because they are easy to use, require no additional hardware, and ensure backward compatibility with existing systems. However, the growing number of cyberattacks and data breaches has led to the introduction of single sign-on (SSO) services, which simplify user management and offload the responsibility of password management from individual servers. While SSO reduces the burden on individual systems, it centralizes password storage, increasing the risk as attackers can now focus on compromising a few critical servers. For instance, in 2023, there were a record-breaking 3,205 data compromises, representing a 72% increase over the previous high recorded in 2021. These breaches exposed approximately 353 million individual records, underscoring the escalating threat of cyberattacks targeting user databases [14].

To address this growing issue, Facebook proposed a system involving a ratelimiter – a cryptographic service that remains oblivious to the password and assists only in its verification. This idea was initially formalized in Pythia [11] and later further developed into Password-Hardening (PH) [20] and Password-Hardened Encryption (PHE) [19]. The key innovation extends beyond simply protecting passwords, introducing a password-based key derivation mechanism. This proposal was quickly commercialized by Virgil Security as part of its software-as-a-service solution, demonstrating its immediate practical relevance and applicability.

Additionally, PH and PHE incorporate the capability for performing key updates without user involvement, a feature not only required by the PCI-DSS standard [26] but also widely recommended as a best

practice by major providers. For example, Google advises regular key rotation to mitigate security risks associated with long-term key usage [13]. Similarly, industry standards such as those from the National Institute of Standards and Technology (NIST) emphasize the importance of key rotation in maintaining robust security practices [4]. Regular key updates enhance security by cryptographically erasing leaked keys and introducing fresh, independent keys.

The rapid adoption of PHE by Virgil Security underscores the urgent need in the industry for solutions that secure password storage while complying with industry standards. However, adapting novel cryptographic primitives to practice always carries the same high risk: novel cryptography is often not well understood, both in terms of the security model and the resulting security of the schemes. In most cases, cryptography only provides security within the model in which it has been proven. If the model deviates significantly from real-world scenarios, the scheme may be insecure against practical attacks. In this work, we show that this is the case for PHE. The original security model evaluates several properties independently and does not consider key rotation as part of all security definitions. We show that this omission leads to a practical cryptographic attack that allows offline brute-force attacks on passwords, the very problem that PHE was designed to prevent. To illustrate the practical implications, we implement the attack using a fork of the PHE implementation by Virgil Security [25]. This attack pushes the state of password hardening back to before the invention of PHE, leaving an important question unanswered:

*Are there efficient Password-based Key Derivation schemes that support rate-limiting and are secure in practice?*

## 1.1 Our Contribution

In this work, we make both negative and positive contributions to the study of password-hardened encryption (PHE):

*Negative Contributions:* We identify a critical security flaw in the formal model of PHE that enables a concrete cryptographic attack on the scheme commercialized by Virgil Security. Our analysis shows that the root cause of this vulnerability lies in a definitional gap: the absence of key rotation as part of some security definitions. To demonstrate the practicality of this attack, we forked the PHE framework of Virgil Security [25] and show how an adversary can (offline) brute-force passwords. We give a comprehensive discussion of the underlying definitional issues and their impact on the concrete attack against Simple PHE [19] and thus on the real-world implementation of Virgil Security in Section 2.

*Positive Contributions:* We address the identified gap by introducing a novel and more realistic security model for PHE, explicitly modeling key rotations as part of the life cycle of a PHE scheme. Building upon this enhanced model, we propose a new PHE design called HildeGUARD that maintains security even with alternating corruptions of the login-server and the ratelimiter. Furthermore, we provide a comprehensive evaluation that underscores both the practicality of the identified attack and the efficiency and robustness of our proposed solution.

This dual approach of identifying flaws and presenting solutions highlights the challenges and opportunities of adapting cryptographic primitives to real-world scenarios.

*Real World Considerations:* Beyond theoretical implications, our work directly impacts PHE deployment. We provide concrete benchmarks and two migration strategies to enable a seamless adoption of HildeGUARD. We show, that the migration from hashed passwords and SimplePHE is efficient, requiring no user re-enrollment; our simulations show the migration of one million users in under a minute. We also describe an opt-out mechanism, ensuring service providers retain the flexibility to transition away from HildeGUARD also without user interaction.

## 1.2 Related Work

Everspaugh et al. [11] initiated the study of password hardening (PH) systems, following an initial proposal by Facebook [22], and introduced constructions based on partially-oblivious pseudorandom functions. Since then, the cryptographic community has proposed several more efficient PH schemes. *PO-COM*, introduced by Schneider et al. [24], suffers from offline attacks in a stronger security model. *Phoenix*, proposed by Lai et al. [20], achieves security under a static corruption model and does not support key derivation capabilities.

In parallel, the concept of password-hardened encryption (PHE) emerged as a natural extension of the PH framework, allowing data to be encrypted under a user’s password while retaining resistance to server compromise. Two PHE schemes are currently known: *Simple PHE*, introduced by Lai et al. [19], and its threshold variant TPHE, proposed by Brost et al. [6]. In this paper, we demonstrate an offline brute-force attack against *Simple PHE*, revealing a critical vulnerability in its design that contradicts its intended security goals. The TPHE construction by Brost et al. [6] is not affected by our attack as their security model considers semi-adaptive corruptions—explicitly capturing the attack scenario that we exploit.

Nonetheless, TPHE’s robustness comes at a significant cost. The protocol requires six rounds of communication and relies heavily on expensive zero-knowledge proofs, making it difficult to deploy in real-world systems where latency, simplicity, and efficiency are paramount. Although one can theoretically instantiate TPHE with a single server to yield a PHE scheme, the associated overhead—both in communication and computation—remains prohibitive. In contrast, our construction is tailored for practical deployment. It achieves strong post-compromise guarantees under a corruption model that reflects the real world, while requiring only a single round of interaction and avoiding costly cryptographic tools where possible.

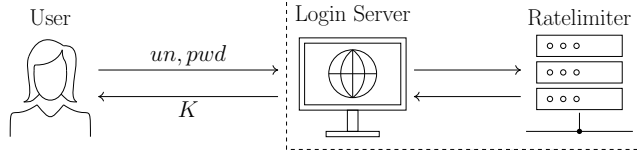
Other Password-based cryptographic primitives include several related terms such as *Password-Protected Secret Sharing (PPSS)*, *Password-Authenticated Key Exchange (PAKE)*, and *Password-Based Threshold Authentication (PbTA)*.

PPSS [3] allows threshold sharing of a secret across servers, requiring a password for retrieval, but lacks key rotation and requires per-user key storage [19,7,16]. In this context, per-user key storage means that each server must store a key for each user it serves. In contrast, the ratelimiter in (T)PH(E) stores only a single key, regardless of the number of users. *Password-Authenticated Key Exchange (PAKE)* and its extensions, such as *Threshold PAKE (T-PAKE)* [3,15] and *Two-Factor Authenticated Key Exchange (TFA-KE)* [17], allow users to establish session keys with servers, but do not support key rotation and also require per-user key storage. *Distributed Password Verification* [8], in an n-out-of-n setting, relies on offline backup tapes for key rotation and also requires per-user keys. *Password-Based Threshold Authentication (PbTA)* [1,5] produces verifiable authentication tokens, but modifies the user-server interface by replacing passwords with tokens and requires direct user interaction with cryptographic servers. Finally, *Practical Password Hardening Based on TLS* [10] is a deployable scheme that uses HMAC with a server’s TLS secret key, but lacks key rotation and is vulnerable to offline dictionary attacks if the server is compromised. DPaSE [9] enables key-derivation based on a password, but it does not support key rotation.

## 2 Technical Outline

We begin by describing the interface used for interaction within the system, as it provides a natural starting point for deriving the intuitive security properties one would expect from such a system. Through this exploration, we show that the Lai et al. security model is inadequate and fails to address specific threats and scenarios that arise in practical deployment. To illustrate the real-world impact of the model’s shortcomings, we outline a high-level attack that exploits these vulnerabilities. Finally, we present our novel scheme that achieves security within this model.

*System outline.* The following description refers to the key-encapsulation variant of PHE as implemented by Virgil Security. PHE involves three participants (cf. Figure 1): the user, who submits a username and password to derive an encapsulated key after providing valid credentials; the login server, which stores an enrollment representing an encryption of the user’s credentials; and the ratelimiter, which validates requests without accessing the user’s secret credentials.



**Fig. 1.** Intuition of password-hardened encryption. The interfaces for the user remain the same, while the key can only be retrieved if both the server and the ratelimiter interact to run the decryption process.

The functional requirements are as follows: The user must remain stateless and perform no cryptographic operations to ensure full compatibility. This constraint also mandates a fixed interface on the login server. To avoid offline brute-force attacks, the login server cannot independently verify the correctness of a password and must interact with the ratelimiter for validation. Besides its own secret, the ratelimiter maintains only a minimal state for counting recent decryption attempts and limiting them. This especially means that the ratelimiter does not keep individual records per user. A critical feature of the system is support for key rotation, which requires sublinear communication between the server and the ratelimiter, while ensuring the user remains uninvolved.

*Security Requirements.* To formulate the desired security properties for PHE, we first consider the single-server setting as the basic model. From there, we incrementally extend the model, ensuring that each extension introduces improved security guarantees. This step-by-step approach allows us to systematically express the incremental improvements in security that each modification provides.

*Single Server Setting.* In the single-server setting, it is obvious that no security can be provided in the event of a server compromise, as all secrets and the password database would be exposed to the attacker. While modern cryptographic techniques such as memory-hard functions [23] can significantly increase the cost of brute-force attacks, they do not fully address this vulnerability. As a result, any security model that aims for the strongest guarantees must exclude server compromise scenarios, since achieving security under such conditions is fundamentally infeasible.

*Two-Server Setting.* To tolerate server compromises, it is necessary to introduce a second component that keeps its own secret. This principle of *distributed trust* is also applied in PHE with the introduction of a *ratelimiter*. Following the principle of *minimal exposure and knowledge*, the ratelimiter never receives the plaintext password or any other private information; instead, it is designed only to assist in the verification process.

In the simplest implementation of this setup, the server computes an HMAC over the password and a nonce, and then sends the resulting value to the ratelimiter. The ratelimiter, in turn, computes another HMAC on this received value and returns the resulting “double HMAC” to the server, which then stores it in the database. Since the output of the HMAC is pseudorandom, the ratelimiter does not gain any information about the password. At the same time, this design ensures that verification requires the active participation of the ratelimiter, since the server alone cannot compute the double HMAC independently. Consequently, even if the password database and the server’s private key are compromised, the stored value remains pseudorandom under the ratelimiter’s key.

The introduction of an additional party, the ratelimiter, inherently creates a new potential attack vector, as an adversary could target this party in addition to the server. However, unlike the single-server setting, the model can now handle corruption of either the server or the ratelimiter independently, since the remaining party’s secret ensures the pseudorandomness of the stored values. The security model should therefore be designed to tolerate the compromise of *either* the server *or* the ratelimiter. However, as in the single-server setting, it is impossible for a scheme to tolerate the compromise of *both* parties simultaneously, since there would be no remaining secret to protect the records storing the password. Thus, the security of this model depends on the assumption that at least one party remains uncompromised at all times.

*Two-Server Setting With Key Rotation.* In practice, it is unrealistic to assume that one of the parties will remain honest at all times. To overcome the impossibility result of the previous setting, we observe that in practice, compromise is often transient: once an intrusion is detected, operators typically respond with containment measures such as credential revocation or privilege reduction, effectively cutting off the attacker’s access to the compromised system. A 2024 survey conducted by Fortinet [12] found that one-third of organizations detected six or more intrusions within a year—highlighting both the frequency of compromises and the feasibility of detecting them. If the secret of the once corrupted party remains unchanged after the adversary loses access, nothing is won: when the adversary corrupts the other party, it has access to both secret states, leaving the records storing the passwords unprotected. This is the case even though the parties are never compromised *simultaneously*.

To address this, various standards and best practices advocate *periodic rotation of cryptographic keys*. However, performing *non-trivial key updates*—where data is re-secured without decrypting and re-encrypting all stored information—poses significant challenges. In our setting, this problem is further complicated by the fact that the password must, to some extent, be part of the update process. For practical reasons, involving the user in these updates is not feasible. In addition, to ensure efficiency, the communication between the server and the ratelimiter during the key update process must remain *independent* of the number of stored records.

In our example, the use of the double-HMAC approach clearly does not support key rotation, as the construction lacks the necessary flexibility to update cryptographic keys without reprocessing all stored values. It appears that achieving efficient and secure key rotation would require the use of *number theoretic properties* or more advanced cryptographic primitives.

From the perspective of the *security model*, the goal is to capture the natural lifecycle of detection, revocation, and restoration that characterizes modern incident response. In other words, we want to tolerate corruption of both the server and the ratelimiter, provided that an honest key update has been performed in the meantime. The underlying intuition is that after a key rotation, the leaked key and the other party’s rotated key would be “out of sync”. This desynchronization ensures that even if one key is compromised, the remaining rotated key can still protect the security of the system.

It is reasonable to assume that an adversary who compromises one party cannot automatically compromise the second. This assumption is rooted in the architectural separation in PHE, where the ratelimiter is designed in a zero-trust fashion to operate independently. It may even be managed by a different organization. As a result, the server and the ratelimiter could rely on distinct hardware, software stacks, and operational policies. This infrastructural and administrative isolation creates a natural barrier to simultaneous compromise—especially considering the narrow time window that may exist between the first compromise and the execution of containment measures, including a key rotation. Moreover, one could schedule periodic, proactive key rotations to anticipate corruption even before it is detected. Therefore, assuming independent corruption is not only theoretically meaningful but also aligned with realistic deployment scenarios.

To formalize this intuition, the security model must explicitly allow the attacker to corrupt one party, trigger an honest key rotation, and then corrupt the other party. Note that the security cannot be restored through a dishonest key rotation, since any state transition controlled or observed by the adversary preserves full knowledge of the party’s internal secrets. De-corruption upon an honest key-rotation is the standard in password-based cryptography [5,6,8,11]. Even in the scenario of alternating corruption, the record storing the password must remain secure; otherwise, the concept of key rotation loses its purpose. This critical aspect is precisely where the Simple PHE model falls short, leading to the break of their scheme. Although this attack is formally outside of their defined model, it highlights the essential need to design security models that are as close to practical considerations as possible. Filling this gap is crucial to ensuring robust security against realistic attack scenarios.

*Breaking Simple PHE.* To describe our offline brute-force attack against Simple PHE [19], we first recall the structure of the enrollment record and then give a detailed step-by-step explanation of the attack. Let  $sk_S$  and  $sk_R$  represent the private keys of the server and the ratelimiter, respectively. The enrollment record, which encodes the password  $pwd$  and hides the encapsulated key  $K$ , is structured as  $T = (t_0, t_1, n_S, n_R)$ ,

where  $n_S$  and  $n_R$  are nonces, and the values  $t_0$  and  $t_1$  are computed as

$$t_0 = H_{S,0}(pwd, n_S)^{sk_S} \cdot H_{R,0}(n_R)^{sk_R}$$

and

$$t_1 = H_{S,1}(pwd, n_S)^{sk_S} \cdot H_{R,1}(n_R)^{sk_R} \cdot K^{sk_S}.$$

We assume that the target user is already enrolled with the secure password **secPW**, and we denote their enrolment record as  $T^{\text{hon}} = (t_0^{\text{hon}}, t_1^{\text{hon}}, n_S^{\text{hon}}, n_R^{\text{hon}})$  with

$$t_0^{\text{hon}} = H_{S,0}(\text{secPW}, n_S^{\text{hon}})^{sk_S} \cdot H_{R,0}(n_R^{\text{hon}})^{sk_R}$$

and

$$t_1^{\text{hon}} = H_{S,1}(\text{secPW}, n_S^{\text{hon}})^{sk_S} \cdot H_{R,1}(n_R^{\text{hon}})^{sk_R} \cdot K^{sk_S}.$$

*Step 1 – Enrolling a Malicious User:* In the first step, the adversary corrupts the ratelimiter and registers a malicious user using a password **malPW**. During the execution of this protocol, it reuses the nonce  $n_R^{\text{hon}}$  used during the enrollment of the target user. The resulting enrollment record  $T^{\text{mal}} = (t_0^{\text{mal}}, t_1^{\text{mal}}, n_S^{\text{mal}}, n_R^{\text{hon}})$  is stored at the honest server. The values  $t_0^{\text{mal}}$  and  $t_1^{\text{mal}}$  are computed as

$$t_0^{\text{mal}} = H_{S,0}(\text{malPW}, n_S^{\text{mal}})^{sk_S} \cdot H_{R,0}(n_R^{\text{hon}})^{sk_R}$$

and

$$t_1^{\text{mal}} = H_{S,1}(\text{malPW}, n_S^{\text{mal}})^{sk_S} \cdot H_{R,1}(n_R^{\text{hon}})^{sk_R} \cdot \hat{K}^{sk_S}.$$

The adversary now releases the ratelimiter<sup>3</sup>, and the system performs an honest key-rotation, turning the malicious ratelimiter into an honest party.

*Step 2 – Corrupting the Server:* The next step is for the adversary to corrupt the server. Note that as a result of the key rotation, the keys of both the server and the ratelimiter have been updated. We refer to the updated keys as  $sk'_S$  and  $sk'_R$  respectively. Since the attacker corrupts the server, it knows  $sk'_S$  and it also knows the password **malPW**, and can extract the ratelimiter part of  $t_0^{\text{mal}}$  as follows

$$H_{R,0}(n_R^{\text{hon}})^{sk'_R} = t_0^{\text{mal}} / H_{S,0}(n_S^{\text{mal}}, \text{malPW})^{sk'_S}.$$

Given that this value is the same in  $t_0^{\text{hon}}$ , the malicious server can extract the hash of the honest user's password from  $t_0^{\text{hon}}$ :

$$H_{S,0}(n_S^{\text{hon}}, \text{secPW}) = \left( t_0^{\text{hon}} / H_{R,0}(n_R)^{sk'_R} \right)^{1/sk'_S}.$$

Given the hash, it is easy to see that the adversary can simply perform an offline brute-force attack. To validate the practical impact of this vulnerability, we implemented the attack using the open-source framework maintained by Virgil Security. Full implementation details and attack traces are provided in Section 5.1.

*A Novel Protocol.* Our protocol shares the idea, similar to the Simple PHE protocol, that the enrolment record consists of two components—an authentication token and the encryption of the encapsulated key. Otherwise, the two protocols are fundamentally different. Below we provide an intuition of the main components, with a full description available in Section 4.

**Protection against malformed ciphertexts:** A critical vulnerability in the original protocol arises from the attacker's ability to reuse his own nonce to “unblind” a target user's enrollment record. To counter this, we introduce a shared nonce, derived jointly by the server and the ratelimiter as  $n \leftarrow H(n_S, n_R)$ . This joint nonce ensures that both parties contribute to the randomness, reducing the risk of malicious reuse or manipulation.

<sup>3</sup> As described above, this is not part of a deliberate tactic of the adversary but rather the consequence of effective incident response.

**Protection against database leakage:** To prevent information leakage from a compromised database, the server encrypts all elements of the enrolment record using a CCA-secure encryption scheme with a symmetric private key,  $sk_S$ . This encryption ensures that even if the database is leaked to a malicious ratelimiter, no information about the cryptographic key or salted password hash is exposed. In addition, the protocol supports key rotation: when the server updates its private key, it locally decrypts and re-encrypts all records. This guarantees that even if the ratelimiter was malicious during the attack, the records remain secure as long as the server’s private key is uncompromised.

**Secure Decryption Protocol:** The decryption process has been fundamentally redesigned to ensure that components of the enrollment record and exchanged messages cannot be reused across multiple instances. This is achieved by incorporating ephemeral keys into the decryption protocol. The ephemeral key introduces session-specific randomness, effectively defeating replay attacks and ensuring that all cryptographic operations remain unique and secure in each instance.

### 3 Password-Hardened Encryption

This section defines the interfaces, explains the security model intuitively, and formally describes the security games.

#### 3.1 Interfaces

We largely follow the work of [19], but change the encryption protocol slightly. From the user’s point of view, the encryption protocol as defined in [19] takes a password and a message as input and stores a ciphertext on the server. In practice, this message is most likely a key for a private key encryption scheme used to encrypt the actual payload. This is how Virgil Security uses PHE [25]. As was done for Password-Protected Secret Sharing [15], we change the interface to reflect this practical use in the theoretical definitions. Instead of taking a message as input to be encrypted, the encryption functionality takes only the user’s password and returns a fresh key and a ciphertext in which the key is stored. We define a password-hardened encryption scheme as follows:

**Definition 1 (Password-Hardened Encryption).** A password-hardened encryption (PHE) scheme, consists of four efficient algorithms ( $\text{Setup}, \text{SKeyGen}, \text{RKeyGen}, \text{Update}$ ) and three efficient protocols ( $\langle \text{S}, \text{R} \rangle_{\text{enc}}, \langle \text{S}, \text{R} \rangle_{\text{dec}}, \langle \text{S}, \text{R} \rangle_{\text{rot}}$ ), that we define as follows:

$pp \leftarrow \text{Setup}(1^\lambda)$ : The setup algorithm takes the security parameter  $1^\lambda$  as input and returns public parameters  $pp$ .

$(pk_S, sk_S) \leftarrow \text{SKeyGen}(pp)$ : The server key generation algorithm takes the public parameter  $pp$  as input and returns a server key pair  $(pk_S, sk_S)$ .

$(pk_R, sk_R) \leftarrow \text{RKeyGen}(pp)$ : The ratelimiter key generation algorithm takes the public parameter  $pp$  as input and returns a ratelimiter key pair  $(pk_R, sk_R)$ . For brevity, we assume that all algorithms and in protocols all parties take  $pp, pk_S$ , and  $pk_R$  as inputs.

$((C, K), \epsilon) \leftarrow \langle \text{S}(sk_S, \text{pwd}), \text{R}(sk_R) \rangle_{\text{enc}}$ : The encryption protocol is run between the server and the ratelimiter. The server takes its secret key  $sk_S$  and the password  $\text{pwd}$  as inputs. The ratelimiter takes its secret key  $sk_R$  as input. The server returns a ciphertext  $C$  and a key  $K$ , while the ratelimiter returns the empty string  $\epsilon$ .

$(K, \epsilon) \leftarrow \langle \text{S}(sk_S, \text{pwd}, C), \text{R}(sk_R) \rangle_{\text{dec}}$ : The decryption protocol is run between the server and the ratelimiter. The server takes its secret key  $sk_S$ , a password  $\text{pwd}$ , and the ciphertext  $C$  as inputs. The ratelimiter takes its secret key  $sk_R$  as input. The server returns a key  $K$  and the ratelimiter returns the empty string  $\epsilon$ .

$((sk'_S, pk'_R, \tau), (sk'_R, pk'_S)) \leftarrow \langle \text{S}(sk_S), \text{R}(sk_R) \rangle_{\text{rot}}$ : The key rotation protocol is run between the server and the ratelimiter. The server and the ratelimiter take their respective secret key  $sk_S$  or  $sk_R$  as input. The server returns its new secret key  $sk'_S$ , the ratelimiter’s updated public key  $pk'_R$ , and an update token  $\tau$ . The ratelimiter returns its new secret key  $sk'_R$  and the server’s updated public key  $pk'_S$ .

$C' \leftarrow \text{Update}(sk_S, \tau, C)$ : The update algorithm takes the server’s secret key  $sk_S$ , the update token  $\tau$ , and the ciphertext  $C$  as inputs. It returns an updated ciphertext  $C'$ .

### 3.2 Security Model

In this section, we close the gap in the existing security definition by Lai *et al.* [19] by proposing enhanced security definitions for PHE. We follow the approach by Brost *et al.* [6] by using the two security definitions, where each definition fully covers one of the security dimensions. Our hiding definition extends the domain of password-hardened encryption to a semi-adaptive corruption model.

**Hiding.** The main limitation of recent security definitions for PHE (cf. [19]) is that they handle different attack vectors in separate games. Partial obliviousness only considers the corruption of the ratelimiter while ignoring the possibility of a corrupt server. Hiding only covers security against a corrupt server while ignoring corrupt ratelimiters. Formally speaking, Lai *et al.* restrict their definition to a static corruption model which implies that an adversary can only ever corrupt a single party but never both, one after the other.

In conclusion, Lai *et al.* assume that all adversaries restrict themselves to exploiting only one attack vector even if a combination of attack vectors would lead to a successful attack. This assumption is unrealistic, as real-world adversaries will exploit all available attack vectors concurrently to accomplish their objectives. As shown in Section 2, this theoretical gap can lead to real-world attacks, like the one we have found against SimplePHE [19]. To close this gap, we follow the approach by Brost *et al.* [6] and merge the security notions of partial obliviousness, hiding, and forward security into a single, comprehensive definition called *hiding*, which allows semi-adaptive corruptions of the server and the ratelimiter. In addition, our definition of hiding offers an important improvement by enforcing rate-limiting, a feature not present in the definition proposed by Brost *et al.*

It is worth noting that we cannot simply remove the threshold-related parts of the TPHE definitions of [6] to obtain definitions for PHE as we have modified the encryption protocol to fit the encapsulation setting. Consequently, we no longer have a distinguishing game where the adversary has to decide which of two messages is encrypted in a ciphertext, but rather a search game where the adversary has to find the uniformly random key encrypted in a ciphertext. This change in comparison to [6] allows for more efficient instantiations while still providing meaningful guarantees for real-world deployments, as in practice, the encrypted message is a uniformly random key anyway.

*Corruption Model.* We require that an honestly executed key rotation must reset the corruption. Therefore, we split the execution of a PH scheme into epochs separated by honest key rotations. During an honest key rotation, all parties are honest. The update token is used only within this honest time frame and safely deleted before any party is corrupted. If the adversary were able to obtain the update token, it could recompute the private state transition of the honest key-rotation, making it a dishonest key rotation that does not reset corruption.

In every epoch, either the server or the ratelimiter can be corrupted. It is important to note that achieving security against a simultaneous corruption of the ratelimiter and the server is generally impossible. Following [6], we focus on semi-adaptive corruption, where the desired corruption has to be announced *before* the next key-rotation, instead of fully adaptive corruption, where corruption can happen at any time. In our security proof, we inject different challenges according to the upcoming corruption during the key rotation. To achieve security under fully adaptive corruption, we would have to guess the corruption beforehand to inject the correct challenge. This reduces the tightness of the proof by a factor of  $(1/2)^{Q_{\text{rot}}}$ , where  $Q_{\text{rot}}$  is the number of key rotations.

*A Corrupted Server.* When the server is corrupted, it trivially learns all passwords and keys for newly generated ciphertexts from the encryption request. Similarly, it learns all passwords used to decrypt ciphertexts and the key if the decryption is successful. However, all remaining passwords should not be brute-forceable by a corrupted server and the keys should remain hidden from it.

*A Corrupted Ratelimiter.* When the ratelimiter is corrupted, it can refuse to answer requests, thus preventing the scheme from working. Nevertheless, it should not learn any password or key used during encryption or decryption and should not be able to brute-force passwords without the server, even if the records are leaked.



Furthermore, a corrupt ratelimiter should not be able to make a decryption attempt successful if an incorrect password is used.

*Enforced Rate-limiting.* Existing security definitions for PHE (and PH) impose an upper limit on the number of decryption attempts for the challenge ciphertext (resp. the challenge enrollment record), even if the ratelimiter (or server) would continue to respond to these requests. This scenario creates a situation where PHE/PH schemes can be proven secure even without any rate-limiting measures implemented within the scheme, because the security game already handles the rate-limiting. In addition, all existing PHE and PH schemes rely solely on the ratelimiter to limit decryption attempts. As a result, a malicious ratelimiter can use unrestricted server interactions to brute-force a password while disregarding its own quota bounds. To address this limitation in current definitions, we allow the adversary to query the decryption oracle arbitrarily often, forcing the scheme to implement rate-limiting measures on both the server and the ratelimiter side.

*The Hiding Game.* We address the above issues in the security game of hiding (Figure 2). The security game samples keys for the server and ratelimiter and draws a random challenge password  $pwd^*$  from a fixed password space  $\mathcal{PW}$ . The challenge password  $pwd^*$  is used in an encryption protocol to generate a ciphertext  $C^*$  and a key  $K^*$ . The adversary wins the security game if it correctly guesses which key is encrypted in the challenge ciphertext  $C^*$ . Furthermore, the adversary can interact with honest parties through multiple oracles which we intuitively describe in the following. We formalize the oracles in Figure 3.

**Encryption & Decryption Oracle:** The oracle runs the corresponding protocol with the adversary, simulating the non-corrupted party. It returns the ciphertext and key or the decrypted key, respectively.

**Challenge Oracles:** If the server is not corrupted during the current epoch, the challenge encryption/decryption oracle calls the corresponding “standard” encryption/decryption oracle on the challenge password. The challenge encryption oracle can only be called once and does not return the challenge key.

**Rotation Oracles:** The honest rotation oracle performs an honest key rotation and resets the corruption state. It returns a secret key according to the adversary’s corruption choice  $\text{Corr}'$  for the next epoch. The dishonest rotation oracle performs a key rotation with the adversary, simulating the non-corrupted party.

**Update Oracle:** The update oracle uses the current update token  $\tau$  to update a ciphertext  $C$  into an updated ciphertext  $C'$  and returns it.

**Definition 2 (Hiding).** *A password-hardened encryption scheme PHE is semi-adaptively hiding if, for any PPT adversary  $\mathcal{A}$ ,  $Q_{\text{val}} \geq 0$ , and any password space  $\mathcal{PW}^4$ , there exists a negligible function  $\text{negl}$ , such that for all  $\lambda \in \mathbb{N}$*

$$\Pr[\text{Hid}_{\text{PHE}, \mathcal{A}, Q_{\text{dec}}, \mathcal{PW}}(\lambda) = 1] \leq \frac{Q_{\text{dec}}}{|\mathcal{PW}|} + \text{negl}(\lambda),$$

where  $Q_{\text{dec}}$  is the rate-limiting threshold of the PHE scheme. The randomness is taken over the random coins of all randomized algorithms. The game

$\text{Hid}_{\text{PHE}, \mathcal{A}, Q_{\text{dec}}, \mathcal{PW}}(\lambda)$  is defined in Figure 2.

**Binding.** Intuitively, binding ensures that a corrupt ratelimiter cannot influence en-/decryption in a way that an incorrect key is obtained. In other words, if decryption returns a key, it is always the same one as returned during encryption, even if the ratelimiter is corrupt. Because the derived key is then used outside of PHE to en-/decrypt data, the importance of always correct keys becomes clearer when looking at the impact there. Assume, for example, that the ratelimiter is temporarily corrupt and deviates from the protocol such that an incorrect key is returned to the user. The user then uses this incorrect key to encrypt data. Now, even if the ratelimiter is honest again, the user cannot obtain the same key to decrypt the data. Consequently, this data is lost forever.

<sup>4</sup> For simplicity, we assume a uniform password distribution. Extending the definition to arbitrary password distributions is straightforward and only requires replacing  $\frac{Q_{\text{dec}}}{|\mathcal{PW}|}$  in the upper bound to a min-entropy term.

| $\text{Hid}_{\text{PHE}, \mathcal{A}, Q_{\text{dec}}, \mathcal{PW}}(\lambda)$ |                                                                                                                               |
|-------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| 1 :                                                                           | $pp \leftarrow \$ \text{Setup}(1^\lambda)$                                                                                    |
| 2 :                                                                           | $(pk_S, sk_S) \leftarrow \$ \text{SKeyGen}(pp), (pk_R, sk_R) \leftarrow \$ \text{RKeyGen}(pp)$                                |
| 3 :                                                                           | $\text{Corr} := \perp, \tau := \epsilon, K^* := \epsilon, \text{pwd}^* \leftarrow \$ \mathcal{PW}$                            |
| 4 :                                                                           | $\mathbb{O} := \{\mathcal{O}_{\text{enc}}, \mathcal{O}_{\text{dec}}, \mathcal{O}_{\text{encCh}}, \mathcal{O}_{\text{decCh}},$ |
| 5 :                                                                           | $\mathcal{O}_{\text{HonRot}}, \mathcal{O}_{\text{DishonRot}}, \mathcal{O}_{\text{Update}}\}$                                  |
| 6 :                                                                           | $K' \leftarrow \mathcal{A}^{\mathbb{O}}(1^\lambda, pk_S, pk_R)$                                                               |
| 7 :                                                                           | <b>return</b> $K' = K^*$                                                                                                      |

**Fig. 2.** The security game of hiding for PHE.

Lai *et al.* [19] and Brost *et al.* [6] even go beyond that and define a stronger notion that they call soundness. Intuitively, soundness requires that a corrupt ratelimiter cannot make the server output an incorrect key for the correct password and vice versa. We argue that this is unnecessarily strong because binding together with hiding already covers the same scenarios as their soundness definitions. We show that all scenarios are covered as intended:

- **Correct password:**
  - **Correct key:** This is the intended behaviour of the protocol.
  - **Incorrect key:** This is not possible as binding ensures that if a key is returned, it is the correct one.
  - **No key:** This is trivially achievable by a corrupt ratelimiter by refusing to answer or by sending an incorrect proof. This scenario is also not covered by the other soundness definitions.
- **Incorrect password:**
  - **Correct key:** This is not possible as otherwise, the corrupt ratelimiter could break the hiding property of the PHE scheme.
  - **Incorrect key:** This is not possible as binding ensures that if a key is returned, it is the correct one.
  - **No key:** This is the intended behaviour of the protocol.

Therefore, we capture the above requirement in a novel definition that we call binding. In the binding game, the adversary is given access to an encryption and a decryption oracle. In both games, the adversary provides all inputs to the server, including the server’s secret key and the used randomness. The adversary wins if it can interact with the honest server in two protocol runs using the same server secret key, ratelimiter public key, and ciphertext but still make the server output two different keys. We formally define binding in Definition 3.

**Definition 3 (Binding).** *A password-hardened encryption scheme PHE is binding if, for any PPT adversary  $\mathcal{A}$  there exists a negligible function  $\text{negl}$ , such that for all  $\lambda \in \mathbb{N}$*

$$\Pr[\text{Bind}_{\text{PHE}, \mathcal{A}}(\lambda) = 1] \leq \text{negl}(\lambda),$$

where the randomness is taken over the random coins of all randomized algorithms. The game  $\text{Bind}_{\text{PH}, \mathcal{A}}(\lambda)$  is defined in Figure 4.

## 4 HildeGUARD

In this section, we propose HildeGUARD, the first round optimal password hardened encryption scheme secure in a semi-adaptive corruption model. One of our primary design goals in developing HildeGUARD was to rely exclusively on well-established cryptographic primitives—specifically, elliptic curves, hash functions, and

|                                                                                                       |                                                                           |
|-------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| $\mathcal{O}_{\text{dec}}(pwd, C)$                                                                    | $\mathcal{O}_{\text{enc}}(pwd)$                                           |
| 1 : $S^* := \text{if } \text{Corr} = S \text{ then } \mathcal{A}$                                     | 1 : $S^* := \text{if } \text{Corr} = S \text{ then } \mathcal{A}$         |
| 2 : $\quad \text{else } S(sk_S, pwd, C)$                                                              | 2 : $\quad \text{else } S(sk_S, pwd)$                                     |
| 3 : $R^* := \text{if } \text{Corr} = R \text{ then } \mathcal{A}$                                     | 3 : $R^* := \text{if } \text{Corr} = R \text{ then } \mathcal{A}$         |
| 4 : $\quad \text{else } R(sk_R)$                                                                      | 4 : $\quad \text{else } R(sk_R)$                                          |
| 5 : $(K, \epsilon) \leftarrow \langle S^*, R^* \rangle_{\text{dec}}$                                  | 5 : $((C, K), \epsilon) \leftarrow \langle S^*, R^* \rangle_{\text{enc}}$ |
| 6 : <b>return</b> $K$                                                                                 | 6 : <b>return</b> $(C, K)$                                                |
| $\mathcal{O}_{\text{decCh}}(C)$                                                                       | $\mathcal{O}_{\text{encCh}}()$                                            |
| 1 : <b>ensure</b> $\text{Corr} \neq S$                                                                | 1 : <b>ensure</b> $\text{Corr} \neq S \wedge K^* = \epsilon$              |
| 2 : $\_ \leftarrow \mathcal{O}_{\text{dec}}(pwd^*, C)$                                                | 2 : $(C^*, K^*) \leftarrow \mathcal{O}_{\text{enc}}(pwd^*)$               |
| 3 : <b>return</b> $\epsilon$                                                                          | 3 : <b>return</b> $C^*$                                                   |
| $\mathcal{O}_{\text{HonRot}}(\text{Corr}')$                                                           |                                                                           |
| 1 : <b>ensure</b> $\text{Corr}' \in \{S, R, \perp\}$                                                  |                                                                           |
| 2 : $((sk_S', pk_R', \tau), (sk_R', pk_S')) \leftarrow \langle S(sk_S), R(sk_R) \rangle_{\text{rot}}$ |                                                                           |
| 3 : $\text{Corr} \leftarrow \text{Corr}'$                                                             |                                                                           |
| 4 : <b>if</b> $\text{Corr} = S$ <b>then return</b> $sk_S'$                                            |                                                                           |
| 5 : <b>if</b> $\text{Corr} = R$ <b>then return</b> $sk_R'$                                            |                                                                           |
| 6 : <b>return</b> $\epsilon$                                                                          |                                                                           |
| $\mathcal{O}_{\text{DishonRot}}()$                                                                    |                                                                           |
| 1 : <b>ensure</b> $\text{Corr} \in \{S, R\}$                                                          |                                                                           |
| 2 : $S^* := \text{if } \text{Corr} = S \text{ then } \mathcal{A} \text{ else } S(sk_S)$               |                                                                           |
| 3 : $R^* := \text{if } \text{Corr} = R \text{ then } \mathcal{A} \text{ else } R(sk_R)$               |                                                                           |
| 4 : $((sk_S', pk_R', \tau), (sk_R', pk_S')) \leftarrow \langle S^*, R^* \rangle_{\text{rot}}$         |                                                                           |
| 5 : <b>return</b> $\epsilon$                                                                          |                                                                           |
| $\mathcal{O}_{\text{Update}}(C)$                                                                      |                                                                           |
| 1 : <b>ensure</b> $\tau \neq \epsilon$                                                                |                                                                           |
| 2 : $C' \leftarrow \text{Update}(sk_S, \tau, C)$                                                      |                                                                           |
| 3 : <b>return</b> $C'$                                                                                |                                                                           |

**Fig. 3.** Oracles for the semi-adaptive PHE hiding definition.

AES encryption. By avoiding more complex or experimental cryptographic constructions, such as pairing-friendly curves, we reduce the risk of subtle implementation flaws and side-channel vulnerabilities that often accompany advanced primitives. This approach allows for a minimalistic codebase that is easier to audit and maintain.

In the following, let  $\mathbb{G}$  be a multiplicative finite cyclic group of order  $q = q(\lambda)$  and  $H_{\{S, R, B, F\}} : \{0, 1\}^* \rightarrow \mathbb{G}$  are hash functions. Let  $\Pi_{\text{Enc}}$  be a CCA-secure private-key encryption scheme. If an assertion fails during the execution of an algorithm, it terminates and returns the special symbol  $\perp$  to indicate an error.

In HildeGUARD, we use a single nonce  $n \leftarrow H_N(ns, nr)$  that combines the randomness contributed by the server and the ratelimiter for rate-limiting. To ensure that decryption attempts are bound to this rate-limit,

| $\text{Bind}_{\text{PHE}, \mathcal{A}}(\lambda)$                                                         | $\mathcal{O}_{\text{enc}}(sk_S, pk_R, pwd, r)$                                                 |
|----------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| 1 : $\text{Queries} := \emptyset$                                                                        | 1 : $\mathcal{S} =: \mathcal{S}(sk_S, pwd^*, pk_R; r)$                                         |
| 2 : $(i, j) \leftarrow_{\$} \mathcal{A}^{\mathcal{O}_{\text{enc}}, \mathcal{O}_{\text{dec}}}(1^\lambda)$ | 2 : $((C, K), \epsilon) \leftarrow_{\$} \langle \mathcal{S}, \mathcal{A} \rangle_{\text{enc}}$ |
| 3 : $(sk_S, pk_R, C, K) := \text{Queries}[i]$                                                            | 3 : $\text{Queries} \stackrel{\cup}{=} \{(sk_S, pk_R, C, K)\}$                                 |
| 4 : $(sk_S', pk_R', C', K') := \text{Queries}[j]$                                                        | 4 : <b>return</b> $\epsilon$                                                                   |
| 5 : $b_0 \leftarrow (sk_S, pk_R, C) = (sk_S', pk_R', C')$                                                | $\mathcal{O}_{\text{dec}}(sk_S, pk_R, pwd, C, r)$                                              |
| 6 : $b_1 \leftarrow (K \neq \perp) \wedge (K' \neq \perp)$                                               | 1 : $\mathcal{S} =: \mathcal{S}(sk_S, pwd^*, C, pk_R; r)$                                      |
| 7 : $b_2 \leftarrow K \neq K'$                                                                           | 2 : $(K, \epsilon) \leftarrow_{\$} \langle \mathcal{S}, \mathcal{A} \rangle_{\text{dec}}$      |
| 8 : <b>return</b> $b_0 \wedge b_1 \wedge b_2$                                                            | 3 : $\text{Queries} \stackrel{\cup}{=} \{(sk_S, pk_R, C, K)\}$                                 |
|                                                                                                          | 4 : <b>return</b> $\epsilon$                                                                   |

**Fig. 4.** PHE binding experiment.

the nonce is used in the computations of the server and the ratelimiter. Besides the nonce, an enrollment record consists of two components  $t_0$  and  $t_1$  that are computed as

$$\begin{aligned} t_0 &\leftarrow \text{H}_R(n, 0)^{sk_R} \cdot \text{H}_S(pwd, n) \\ t_1 &\leftarrow \text{H}_R(n, 1)^{sk_R} \cdot K. \end{aligned}$$

The ratelimiter parts of these computations  $h_{R,0} = \text{H}_R(n, 0)^{sk_R}$  and  $h_{R,1} = \text{H}_R(n, 1)^{sk_R}$  can be thought of as an PRF evaluation on the nonce  $\text{PRF}_{sk_R}^{\{0,1\}}(n)$ . Consequently,  $t_0$  is—on an intuitive level—the first ratelimiter PRF multiplied by the salted hash of the password, and  $t_1$  the second ratelimiter PRF multiplied by the encapsulated key. From the server’s perspective, both values are uniformly random because of the pseudorandom values computed by the ratelimiter. To ensure that the record does not leak anything to the ratelimiter, the whole record has to be encrypted by the server, because the ratelimiter knows the corresponding PRF key and consequently also the PRF values.

The decryption protocol can be thought of as a password-authenticated key exchange protocol (PAKE), where the key exchange only succeeds if both parties enter the same password. In contrast to standard PAKE, the server and the ratelimiter enter the first ratelimiter PRF value  $h_{R,0}$  instead of the password. While the ratelimiter can directly compute  $h_{R,0}$ , the server has to use the correct password to extract the PRF value from the stored record  $h'_{R,0} \leftarrow t_0 / \text{H}_S(pwd', n)$ . The ratelimiter uses the resulting key to encrypt the second ratelimiter PRF value  $h_{R,1}$  before sending it to the server. If the entered password is correct, the server obtains the correct decryption key and can decrypt  $h_{R,1}$  to deblind  $t_1$ , yielding  $K$ .

#### 4.1 Description of HildeGUARD

*Encryption Protocol.* To generate a new cryptographic key  $K$  protected by the password  $pwd$ , the server begins by sampling a random nonce  $n_S$  and sending it to the ratelimiter. The ratelimiter also samples a random nonce  $n_R$  and derives a combined nonce  $n \leftarrow \text{H}_N(n_S, n_R)$ . This unified nonce is one of the countermeasures mitigating malformed ciphertexts when using a semi-adaptive corruption model. To proceed, the ratelimiter evaluates two PRFs on  $n$ , using domain separation via a single bit: it computes  $h_{R,0} \leftarrow \text{H}_R(n, 0)^{sk_R}$  and  $h_{R,1} \leftarrow \text{H}_R(n, 1)^{sk_R}$ . Finally, it constructs a Schnorr proof of well-formedness of  $h_{R,0}$  and  $h_{R,1}$  and sends the PRF values alongside the nonce-share  $n_R$ , and the proof  $\pi$  to the server. Receiving these values, the server validates the proof  $\pi$ . As described above,  $h_{R,0}$  is used to blind the salted hash of the password, while  $h_{R,1}$  is used to blind the cryptographic key  $K$ . Consequently, the values  $t_0$  and  $t_1$  are computed as

$$\begin{aligned} t_0 &\leftarrow \text{H}_R(n, 0)^{sk_R} \cdot \text{H}_S(pwd, n) \\ t_1 &\leftarrow \text{H}_R(n, 1)^{sk_R} \cdot K. \end{aligned}$$

| Setup( $1^\lambda$ )                                                  | SKeyGen( $pp$ )                                          | RKeyGen( $pp$ )                  |
|-----------------------------------------------------------------------|----------------------------------------------------------|----------------------------------|
| $g \leftarrow \mathbb{G}$                                             | $sk_S \leftarrow \Pi_{\text{Enc}}.\text{Gen}(1^\lambda)$ | $sk_R \leftarrow \mathbb{Z}_q^*$ |
| $crs \leftarrow \Pi_{\text{NIZK}}(1^\lambda)$                         | $pk_S \leftarrow \epsilon$                               | $pk_R \leftarrow g^{sk_R}$       |
| $H_{\{S,R,B,F\}} \leftarrow \{H : \{0,1\}^* \rightarrow \mathbb{G}\}$ | <b>return</b> ( $pk_S, sk_S$ )                           | <b>return</b> ( $pk_R, sk_R$ )   |
| $H_N \leftarrow \{H : \{0,1\}^* \rightarrow \{0,1\}^\lambda\}$        |                                                          |                                  |
| <b>return</b> $pp \leftarrow (g, crs, H_{\{S,R,B,F,N\}})$             |                                                          |                                  |

**Fig. 5.** The setup algorithms of HildeGUARD.

Finally, the server encrypts the record  $(t_0, t_1, n)$  using its secret key  $sk_S$  and stores the ciphertext. It is important to note that the encapsulated key is a group element. Before using it outside of HildeGUARD, it should be processed by a key derivation function to ensure that the key is a uniformly random bit string. We formally describe the encryption protocol in Figure 6.

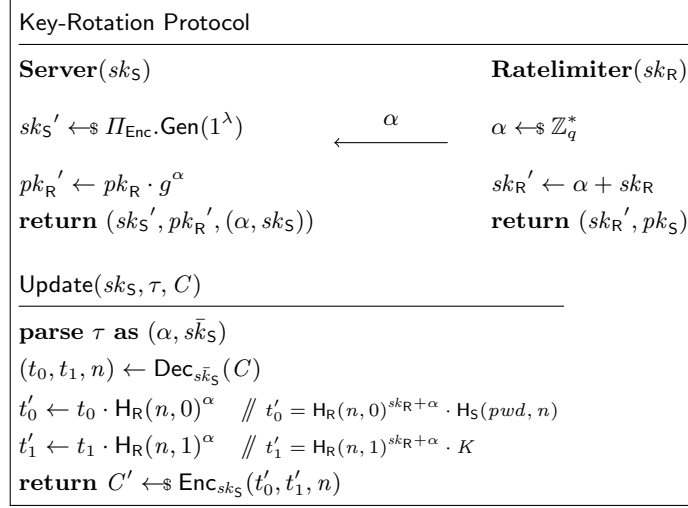
| Encryption Protocol                                                      |                                                                          |
|--------------------------------------------------------------------------|--------------------------------------------------------------------------|
| <b>Server</b> ( $sk_S, pwd$ )                                            | <b>Ratelimiter</b> ( $sk_R$ )                                            |
| $K \leftarrow \mathbb{G}, n_S \leftarrow \{0,1\}^\lambda$                | $n_R \leftarrow \{0,1\}^\lambda, n \leftarrow H_N(n_S, n_R)$             |
|                                                                          | $h_{R,0} \leftarrow H_R(n, 0)^{sk_R}$                                    |
|                                                                          | $h_{R,1} \leftarrow H_R(n, 1)^{sk_R}$                                    |
|                                                                          | $stmt \leftarrow (H_R(n, 0), h_{R,0},$<br>$H_R(n, 1), h_{R,1}, g, pk_R)$ |
| $n \leftarrow H_N(n_S, n_R)$                                             | $(h_{R,0}, h_{R,1}, n_R, \pi) \leftarrow \text{Prove}(crs, stmt, sk_R)$  |
| $stmt \leftarrow (H_R(n, 0), h_{R,0},$<br>$H_R(n, 1), h_{R,1}, g, pk_R)$ |                                                                          |
| <b>ensure</b> $\text{Verify}(crs, stmt, \pi) = 1$                        |                                                                          |
| $h_S \leftarrow H_S(pwd, n)$                                             |                                                                          |
| $t_0 \leftarrow h_{R,0} \cdot h_S$                                       |                                                                          |
| $\parallel t_0 = H_R(n, 0)^{sk_R} \cdot H_S(pwd, n)$                     |                                                                          |
| $t_1 \leftarrow h_{R,1} \cdot K$                                         |                                                                          |
| $\parallel t_1 = H_R(n, 1)^{sk_R} \cdot K$                               |                                                                          |
| $C \leftarrow \text{Enc}_{sk_S}(t_0, t_1, n)$                            |                                                                          |
| <b>return</b> ( $C, K$ )                                                 |                                                                          |

**Fig. 6.** The encryption protocol of HildeGUARD.

*Decryption Protocol.* The decryption protocol serves two primary purposes: it authenticates the user by verifying the candidate password and, upon successful authentication, provides the cryptographic key  $K$  to the user. To defend against online brute-force attacks, both the server and the ratelimiter enforce rate-limiting via a **ratelimit** function that tracks the number of decryption attempts per user. This function is initialized with a fixed quota  $Q_{\text{val}}$  and aborts the protocol if the limit is exceeded, thereby blocking excessive login attempts.



PRF values consistent with the new ratelimiter key  $sk_R' = sk_R + \alpha$ . It re-encrypts the resulting values  $t'_0, t'_1, n$  with its new encryption key. We formalize the key-rotation protocol and update algorithm in Figure 8.



**Fig. 8.** The key rotation protocol of HildeGUARD.

## 4.2 Security of HildeGUARD

We give an intuition on why HildeGUARD is secure and defer the formal proofs to Appendix A.

*Hiding.* We break down the lifetime of a HildeGUARD instance into epochs, where the server's and ratelimiter's keys of an epoch are independent of the keys of other epochs. We distinguish two cases based on the corruption.

If the server is corrupt, it learns nothing from  $t_0$  and  $t_1$  because of the pseudorandomness of  $h_{R,0}$  and  $h_{R,1}$ . Furthermore, we show that the adversary can validate only a single password guess per decryption protocol run with the honest ratelimiter. If it could compute  $h_B^{r_B}$  for multiple  $h_B$  to test multiple password guesses, it breaks the OM-gapDH assumption.

If the ratelimiter is corrupt, it learns nothing from the ciphertext because of the CCA-security of the used symmetric-key encryption scheme. From interactions with the honest server in the decryption protocol, it only learns if an entered password was correct from the output of the server. Beyond that, it learns nothing because the value  $x$  is uniformly random due to the blinding with  $r_S$ .

The **ratelimit** function restricts the number of decryption protocol runs with honest parties, such that the adversary can test at most  $Q_{\text{limit}}$  passwords.

*Binding.* Proving that HildeGUARD is binding is relatively straightforward. If the adversary interacts with an honest server in two protocol runs involving the same ciphertext but different keys, we can extract two NIZKs that have contradicting statements. Because the same ciphertext is used, we know that also the same  $t_1 = h_{R,1} \cdot K$  and  $n$  is used by the server. The two different keys  $K$  and  $K'$  lead to two different ratelimiter PRF values  $h_{R,1} \leftarrow t_1/K$  and  $h'_{R,1} \leftarrow t_1/K'$ . The NIZKs provided by the adversary prove that both  $h_{R,1} \neq h'_{R,1}$  are well-formed, which cannot be true. These NIZKs break the soundness of the NIZK scheme.

## 5 Impact

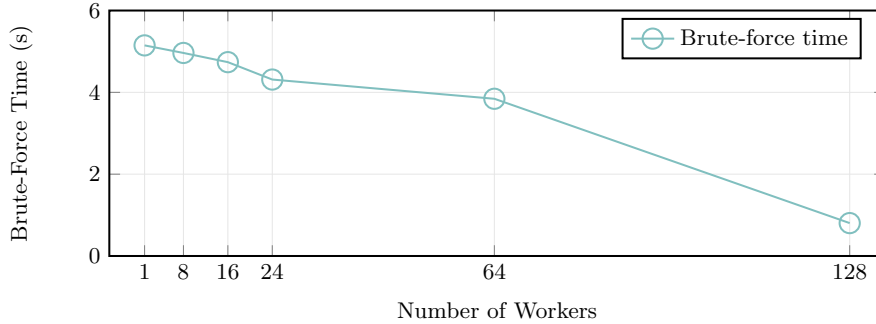
We support our work with implementations and benchmarks, demonstrating both the feasibility of our attack and the efficiency of our proposed protocol, HildeGUARD. We begin by demonstrating and benchmarking our attack against simplePHE and Virgil Security’s implementation. Then, we implement HildeGUARD and benchmark its efficiency, comparing it to SimplePHE.

### 5.1 Breaking SimplePHE

To demonstrate the feasibility of our attack while adhering to ethical standards, we cloned the latest version of Virgil Security’s PHE implementation [25] and conducted the attack on a local machine. The implementation, written in Golang [2], was executed using Go version 1.23.4 on a MacBook Pro equipped with an M3 Pro chip and 36GB of unified memory, running macOS Sonoma 14.6.1.

The attack involves a malicious ratelimiter reusing a nonce from an honest user’s enrollment record during the login of a user with a known password. After performing a key rotation and compromising the server, the attacker uses the malicious password and login record to extract the ratelimiter’s PRF. Using the extracted PRF, the attacker computes the salted hash of the honest user’s password and initiates a brute-force attack to recover it. This attack has been implemented and validated on the described system configuration. We refer the reader to Section 2, a full description of which is given in the paragraph *Breaking Simple PHE*.

*Benchmark of our attack.* For testing the feasibility of our attack, we used a password list from SecLists containing 10 million passwords [21] and selected the secPW uniformly out of it. The brute-force attack successfully recovered the honest user’s password in all test cases. We evaluated the attack using different levels of parallelism, varying the number of concurrent workers. The results are summarized in Figure 9.



**Fig. 9.** Brute-force attack time with varying worker counts

Our results show that using 128 workers provides a significant speedup, completing the brute-force attack in approximately 804 ms. However, even with a single worker, the attack remains practical, recovering the password in just over 5 seconds. We publish the implementation of our attack for review purposes. For an ethical discussion on this matter, we refer to Section 7.

### 5.2 Benchmark of HildeGUARD

We benchmark the performance of HildeGUARD against the existing practical PHE scheme Simple PHE [19]. We do not compare the performance compared to TPHE [6], since this work uses three rounds which makes it highly unpractical. In addition, the spare latency induced by the number of rounds makes TPHE meaningless as comparison.

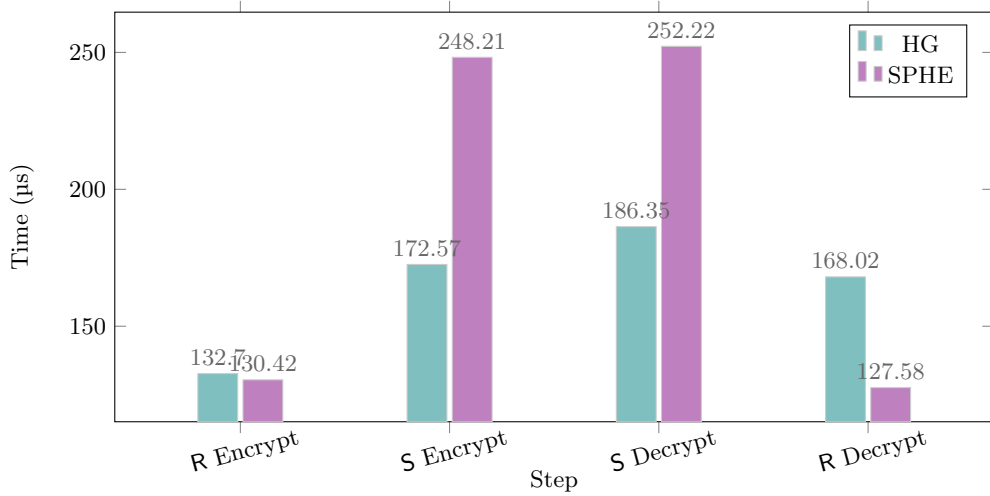
Our implementation is written in Rust and compiled using `rustc` 1.84.0. It utilizes `curve25519-dalek` 4.1.3 for elliptic curve operations and AES-GCM for encryption.



*Cryptographic Benchmark Results.* To evaluate the efficiency of our implementation, we benchmark HildeGUARD against SimplePHE across the cryptographic operations. Our tests were conducted on a MacBook Pro with an M3 Pro chip and 36GB of unified memory, running macOS Sonoma 14.6.1. The benchmarking framework is powered by Criterion, and multi-threading is managed using `tokio` with `rt-multi-thread` enabled. Table 1 and Figure 10 present the execution times (in microseconds) for HildeGUARD and SimplePHE across different cryptographic steps.

**Table 1.** Cryptographic Benchmark Comparison: HildeGUARD vs. SimplePHE. Time is measured in  $\mu\text{s}$ .

| Step             | HildeGUARD | SimplePHE |
|------------------|------------|-----------|
| R.encrypt        | 132.7      | 130.42    |
| S.encrypt_init   | 5.74       | -         |
| S.encrypt_finish | 166.83     | 248.21    |
| S.decrypt_init   | 43.57      | 38.98     |
| R.decrypt        | 168.02     | 127.58    |
| S.decrypt_finish | 142.78     | 213.24    |



**Fig. 10.** Filtered Cryptographic Benchmark: HildeGUARD vs. SimplePHE

Encryption results show that `R.encrypt` performs similarly in both schemes (132.7  $\mu\text{s}$  for HildeGUARD vs. 130.42  $\mu\text{s}$  for SimplePHE). However, `S.encrypt_finish` in HildeGUARD is faster (166.83  $\mu\text{s}$  vs. 248.21  $\mu\text{s}$ ), reducing the total encryption time. For decryption, `S.decrypt_init` is identical for both schemes (43.57  $\mu\text{s}$  vs. 38.98  $\mu\text{s}$ ), while `S.decrypt_finish` is faster (142.78  $\mu\text{s}$  vs. 213.24  $\mu\text{s}$ ). `R.decrypt` is slower in HildeGUARD (168.02  $\mu\text{s}$  vs. 127.58  $\mu\text{s}$ ), leading to a small overhead at the ratelimiter. Overall, HildeGUARD reduces the computational cost of server-side encryption and decryption but introduces some additional cost at the ratelimiter. However, these differences do not impact authentication latency as we explore in the online performance benchmark.

*Online Performance.* To evaluate encryption and decryption latencies in a real-world setting, we deployed our prototype on AWS EC2 `c5.2xlarge` instances, equipped with an Intel(R) Xeon(R) Platinum 8000 series processor featuring eight virtual CPUs and 16 GiB of memory. The servers run an Actix web server

(version 4) to handle requests. Performance evaluations were conducted under varying network latencies. We measure latencies using Apache Benchmark 2.3 and report averages over 1000 interactions in Table 2. The validation time for HildeGUARD is 50.3 ms, while SimplePHE achieves a slightly lower value of 49.668 ms. The observed differences are beyond the variance introduced by network latency, which aligns with our analysis of the cryptographic core.

To isolate the cryptographic overhead, we also benchmarked a “dummy implementation” where the web server simply responds to requests without performing any cryptographic operations. This dummy benchmark includes one round of communication between the server and the ratelimiter. Interestingly, some validation times for SimplePHE and HildeGUARD are slightly lower than the dummy’s 50.429 ms response time. This confirms that the cryptographic operations introduce negligible overhead compared to the overall request handling and network communication. Both schemes share the same round complexity, meaning that differences in execution time primarily stem from cryptographic operations rather than additional protocol steps.

**Table 2.** Online Performance Benchmark: HildeGUARD vs. SimplePHE. Time is measured in milliseconds.

| Scheme     | Enroll (ms) | Validate (ms) |
|------------|-------------|---------------|
| HildeGUARD | 49.789      | 50.3          |
| SimplePHE  | 50.758      | 49.668        |
| Dummy      | 50.429      |               |

### 5.3 Deployment

We have demonstrated an attack against SimplePHE and introduced HildeGUARD as a more secure alternative. For systems where SimplePHE is already deployed, we provide a seamless migration strategy to transition ciphertexts from SimplePHE to HildeGUARD without requiring password re-enrollment or user interaction. Additionally, we describe how HildeGUARD can be integrated into systems that currently use salted hashes for password security. This ensures compatibility with existing authentication frameworks while enhancing security. Finally, we address a critical limitation of PHE schemes: their reliance on a specific ratelimiter, which undermines robustness. If the ratelimiter’s service becomes unavailable—whether due to termination, service disruptions, or infrastructure changes—all ciphertexts stored on the server become unusable. To enhance system robustness and resilience, we propose an opt-out mechanism that allows a seamless transition from HildeGUARD back to traditional salted and hashed passwords. This approach ensures that deployments can switch authentication methods or replace the ratelimiter without compromising the usability of existing records.

*Migration from Salted Hash.* We demonstrate how to migrate an existing authentication system from hashed passwords to HildeGUARD while preserving compatibility with legacy storage. Inspired by Facebook’s *Password Onion* [22], we use the stored salted and hashed password as the input password for our PHE scheme. Besides that, the HildeGUARD encryption protocol stays as is.

Our experiments validate the practicality of adopting HildeGUARD without user involvement. To measure the computational overhead of this migration, we benchmarked the enrollment and verification process of one million hashed passwords. The results, summarized in Table 3, show that encryption–mapping hashed passwords to HildeGUARD-protected ciphertexts—completed in 48.18 seconds, averaging 0.0482 ms per user.

Importantly, this initial migration can be performed locally by the server without any interaction with the future ratelimiter. During the process, the server generates the ratelimiter’s secret key and performs the rate limiter’s part of the encryption process locally. Once the migration is complete, the server securely transfers the secret key to the ratelimiter and erases it from local storage. This temporary exposure of the ratelimiter’s secret key to the server does not introduce any additional security risks. At this stage, a malicious server

could already copy the unprotected database and perform a dictionary attack, as the data has not yet been protected by the ratelimiter.

**Table 3.** Migrating one million users from salted hash to HildeGUARD

|              | Operation Total (s) | Average per User (ms) |
|--------------|---------------------|-----------------------|
| Encryption   | 48.181              | 0.0482                |
| Verification | 53.172              | 0.0532                |

*Opt-Out.* The primary advantage of PHE is its enhanced security for users. However, in practice, service providers require flexibility to adapt their authentication mechanisms over time. Locking in on a single cryptographic scheme can hinder long-term maintainability. To address this, we introduce opt-out for PHE, a method for transitioning from PHE-protected records back to traditional salted hashes. This follows a concept that Jia *et al.* introduced for PH [18].

Opt-out enables providers to migrate to a different authentication scheme or replace the ratelimiter without requiring user interaction. Opt-out takes place when the ratelimiter’s service should be discarded. Therefore, we assume a service provider knows both the server’s key and the ratelimiter’s key. Even though this might seem like a severe security risk that renders the whole scheme useless, we argue why it is reasonable to do so. First of all, the sole purpose of the opt-out functionality is to release the ratelimiter from its duty, enabling the server to verify passwords on its own. Therefore, we no longer need the security that the secrecy of the ratelimiter’s secret key provides. Furthermore, the opt-out protocol is executed only after the server and the ratelimiter agree to proceed through an out-of-band process.

$$\begin{aligned} H_S(pwd, n) &\leftarrow t_0/h_{R,0} \\ K &\leftarrow t_1/h_{R,1}. \end{aligned}$$

The resulting salted hash of the password is used to compute two new tokens. One is solely for verifying an entered password, while the other one stores the key. To achieve domain separation,  $h_S$  is hashed with a single bit as a domain separator. The first token consists only of this hash  $t'_0 \leftarrow H(h_S, 0)$ , while the second token is the key blinded by the second hash  $t'_1 \leftarrow H(h_S, 1) \cdot K$ . The formal definition of the opt-out algorithm can be seen in Figure 11.

It is essential that the opt-out procedure does not expose passwords directly. Furthermore, the resulting enrollment record must offer at least the same level of protection against dictionary attacks as a standard salted password hash.

Given a stored HildeGUARD record, the opt-out process proceeds as follows. The service provider first reconstructs the ratelimiter’s contribution to the ciphertext by computing the PRF outputs:

$$\begin{aligned} t'_0 &\leftarrow t_0/h_{R,0} = H_S(pwd, n) \\ t'_1 &\leftarrow t_1/h_{R,1} = K. \end{aligned}$$

These values are used to obtain the hashed password and the encapsulated key from the ciphertext:

$$\begin{aligned} H_S(pwd, n) &\leftarrow t'_0/h_{R,0} \\ K &\leftarrow t'_1/h_{R,1}. \end{aligned}$$

The resulting salted password hash  $H_S(pwd, n)$  is used to derive two separate tokens: one for password verification and another for storing the cryptographic key. Domain separation is achieved by hashing  $h_S$  with a distinguishing bit. Specifically, the first token is  $t'_0 \leftarrow H(h_S, 0)$ , and the second token is  $t'_1 \leftarrow H(h_S, 1) \cdot K$ , which conceals the key. The formal description of the opt-out algorithm is provided in Figure 11. We benchmarked this opt-out procedure for HildeGUARD and the opt-out completes in  $63.92\mu s$  per ciphertext on average.

|                                                                                                                                                                                                                                                                                                                         |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Opt-Out</b> ( $sk_S, sk_R, C$ )<br><hr/> $(t_0, t_1, n) \leftarrow \text{Dec}_{sk_S}(C)$<br>$h_S \leftarrow t_0 / H_R(n, 0)^{sk_R}, K \leftarrow t_1 / H_R(n, 1)^{sk_R}$<br>$t'_0 \leftarrow H(h_S, 0), t'_1 \leftarrow H(h_S, 1) \cdot K$<br>$C' \leftarrow \text{Enc}_{sk_S}(t'_0, t'_1, n)$<br><b>return</b> $C'$ |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Fig. 11.** The opt-out algorithm of HildeGUARD.

*Migration from SimplePHE.* The opt-out mechanism not only allows migration from HildeGUARD to traditional salted hashes but also enables a seamless transition from existing PHE schemes, such as SimplePHE, to HildeGUARD. This is crucial for providers seeking to upgrade their security model without requiring user action.

To migrate from SimplePHE, the provider first applies the opt-out procedure to extract the salted hash of the password. This process follows the same steps as transitioning from HildeGUARD to salted hashes: computing the ratelimiter’s PRF values, stripping the ratelimiter’s contribution, and recovering the hashed password. Once obtained, this salted hash is used as the password input for HildeGUARD, ensuring compatibility with existing credentials. Re-enrollment into HildeGUARD follows the standard encryption workflow, using the extracted salted hash in place of a plaintext password.

## 6 Conclusion

In this work, we revisited the security of Password-Hardened Encryption (PHE) and identified a critical vulnerability in its formal model. We demonstrated a practical attack that allows offline brute-force recovery of passwords, highlighting the necessity of a more comprehensive security framework. Our findings emphasize that security definitions must align with real-world deployment scenarios, particularly in the presence of key rotations.

To address these shortcomings, we introduced HildeGUARD, a novel PHE scheme that remains secure even under adaptive corruptions. We formalized a refined security model, ensuring robustness against the identified attack while maintaining efficiency. Our implementation and benchmarks demonstrate that HildeGUARD not only mitigates the vulnerabilities of SimplePHE but also achieves comparable performance.

Beyond theoretical contributions, we provide a migration strategy for transitioning existing PHE deployments to HildeGUARD and offer integration pathways for systems relying on traditional salted hashes. Our work underscores the importance of continuous scrutiny in cryptographic protocol design, ensuring that practical deployments uphold security guarantees against evolving threats.

## Open Science

To facilitate transparency, reproducibility, and further research, we publicly release our attack and prototype implementations. The full codebase is available at:

<https://github.com/pGerhart/Attack-SimplePHE>  
<https://github.com/pGerhart/HildeGUARD>  
<https://github.com/pGerhart/SimplePHE>

## 7 Ethics Considerations

This research adheres to established ethical principles in cybersecurity research. We do not exploit vulnerabilities in deployed systems or compromise real user data. Instead, our work identifies a design flaw in Virgil Security’s PHE protocol and demonstrates its feasibility in a controlled environment.

*Responsible Disclosure.* Following ethical best practices, we engaged in responsible disclosure with Virgil Security prior to publication. We provided a detailed report outlining the attack, its security implications, and possible mitigations. This ensured that the vendor had an opportunity to address the issue before public disclosure, aligning with the principles of coordinated vulnerability disclosure.

*Compliance with the Menlo Report.* Our study aligns with the ethical principles set forth in the Menlo Report, which extends traditional research ethics to the field of cybersecurity:

- **Respect for Persons:** Our research does not target real users or deployed systems. All experiments were conducted on a locally controlled instance of the protocol.
- **Beneficence:** The goal of our work is to strengthen security, not to facilitate attacks. By responsibly disclosing our findings, we help prevent potential misuse.
- **Justice:** The identified vulnerability has broad implications for users relying on PHE-based authentication. Our research ensures that security risks are addressed equitably.
- **Respect for Law and Public Interest:** We operate within legal and ethical boundaries, following responsible disclosure procedures and ensuring that no unauthorized access occurs.

*Post-Publication Plans.* We publicly release our implementation for verification and transparency. However, we do not provide a direct exploit, preventing misuse while enabling security researchers and vendors to evaluate and improve their implementations. By adhering to these ethical guidelines, we ensure that our work contributes to the security of cryptographic protocols without introducing harm or facilitating real-world attacks.

## Acknowledgments

This work was partially supported by Deutsche Forschungsgemeinschaft as part of the Research and Training Group 2475 “Cybercrime and Forensic Computing” (grant number 393541319/GRK2475/1-2019) and through grant 442893093, and by the Smart Networks and Services Joint Undertaking (SNS JU) under the European Union’s Horizon Europe research and innovation program in the scope of the CONFIDENTIAL6G project under Grant Agreement 101096435. The contents of this publication are the sole responsibility of the authors and do not in any way reflect the views of the EU. This work is also partially supported by SBA Research (SBA-K1 NGC), which is a COMET Center within the COMET – Competence Centers for Excellent Technologies Programme and funded by BMIMI, BMWET, and the federal state of Vienna. The COMET Programme is managed by FFG.

## References

1. Shashank Agrawal, Peihan Miao, Payman Mohassel, and Pratyay Mukherjee. PASTA: PASsword-based threshold authentication. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018*, pages 2042–2059. ACM Press, October 2018.
2. The Go Authors. The go programming language. <https://go.dev>, 2025.
3. Ali Bagherzandi, Stanislaw Jarecki, Nitesh Saxena, and Yanbin Lu. Password-protected secret sharing. In Yan Chen, George Danezis, and Vitaly Shmatikov, editors, *ACM CCS 2011*, pages 433–444. ACM Press, October 2011.
4. Elaine Barker. Nist special publication 800-57 part 1, revision 5. <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-57pt1r5.pdf>. [Online; accessed 21-January-2025].
5. Carsten Baum, Tore Kasper Frederiksen, Julia Hesse, Anja Lehmann, and Avishay Yanai. PESTO: Proactively secure distributed single sign-on, or how to trust a hacked server. In *2020 IEEE European Symposium on Security and Privacy*, pages 587–606. IEEE Computer Society Press, September 2020.
6. Julian Brost, Christoph Egger, Russell W. F. Lai, Fritz Schmid, Dominique Schröder, and Markus Zoppelt. Threshold password-hardened encryption services. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020*, pages 409–424. ACM Press, November 2020.

7. Jan Camenisch, Anja Lehmann, Anna Lysyanskaya, and Gregory Neven. Memento: How to reconstruct your secrets from a single password in a hostile environment. In Juan A. Garay and Rosario Gennaro, editors, *CRYPTO 2014, Part II*, volume 8617 of *LNCS*, pages 256–275. Springer, Berlin, Heidelberg, August 2014.
8. Jan Camenisch, Anja Lehmann, and Gregory Neven. Optimal distributed password verification. In Indrajit Ray, Ninghui Li, and Christopher Kruegel, editors, *ACM CCS 2015*, pages 182–194. ACM Press, October 2015.
9. Poulami Das, Julia Hesse, and Anja Lehmann. DPaSE: Distributed password-authenticated symmetric-key encryption, or how to get many keys from one password. In Yuji Suga, Kouichi Sakurai, Xuhua Ding, and Kazuo Sako, editors, *ASIACCS 22*, pages 682–696. ACM Press, May / June 2022.
10. Constantinos Diomedous and Elias Athanasopoulos. Practical password hardening based on tls. In Roberto Perdisci, Clémentine Maurice, Giorgio Giacinto, and Magnus Almgren, editors, *Detection of Intrusions and Malware, and Vulnerability Assessment*, pages 441–460, Cham, 2019. Springer International Publishing.
11. Adam Everspaugh, Rahul Chatterjee, Samuel Scott, Ari Juels, and Thomas Ristenpart. The pythia PRF service. In Jaeyeon Jung and Thorsten Holz, editors, *USENIX Security 2015*, pages 547–562. USENIX Association, August 2015.
12. Fortinet. 2024 state of operational technology and cybersecurity report. Technical report, Fortinet, June 2024. Accessed: 2025-05-07.
13. Google. Google cloud kms documentation: Key rotation. <https://cloud.google.com/kms/docs/key-rotation>. [Online; accessed 21-January-2025].
14. Identity Theft Resource Center. 2023 annual data breach report reveals record number of compromises, 72 percent increase over previous high. 2023. Accessed: 2025-01-21.
15. Stanislaw Jarecki, Aggelos Kiayias, and Hugo Krawczyk. Round-optimal password-protected secret sharing and T-PAKE in the password-only model. In Palash Sarkar and Tetsu Iwata, editors, *ASIACRYPT 2014, Part II*, volume 8874 of *LNCS*, pages 233–253. Springer, Berlin, Heidelberg, December 2014.
16. Stanislaw Jarecki, Aggelos Kiayias, Hugo Krawczyk, and Jiayu Xu. TOPPSS: Cost-minimal password-protected secret sharing based on threshold OPRF. In Dieter Gollmann, Atsuko Miyaji, and Hiroaki Kikuchi, editors, *ACNS 17 International Conference on Applied Cryptography and Network Security*, volume 10355 of *LNCS*, pages 39–58. Springer, Cham, July 2017.
17. Stanislaw Jarecki, Hugo Krawczyk, Maliheh Shirvanian, and Nitesh Saxena. Two-factor authentication with end-to-end password security. In Michel Abdalla and Ricardo Dahab, editors, *PKC 2018, Part II*, volume 10770 of *LNCS*, pages 431–461. Springer, Cham, March 2018.
18. C. Jia, S. Wu, and D. Wang. Reliable password hardening service with opt-out. In *2022 41st International Symposium on Reliable Distributed Systems (SRDS)*, pages 250–261, Los Alamitos, CA, USA, sep 2022. IEEE Computer Society.
19. Russell W. F. Lai, Christoph Egger, Manuel Reinert, Sherman S. M. Chow, Matteo Maffei, and Dominique Schröder. Simple password-hardened encryption services. In William Enck and Adrienne Porter Felt, editors, *USENIX Security 2018*, pages 1405–1421. USENIX Association, August 2018.
20. Russell W. F. Lai, Christoph Egger, Dominique Schröder, and Sherman S. M. Chow. Phoenix: Rebirth of a cryptographic password-hardening service. In Engin Kirda and Thomas Ristenpart, editors, *USENIX Security 2017*, pages 899–916. USENIX Association, August 2017.
21. Daniel Miessler and contributors. Seclists: Common-credentials 10 million password list (top 1,000,000). <https://github.com/danielmiessler/SecLists/blob/master/Passwords/Common-Credentials/10-million-password-list-top-1000000.txt>, 2025. Accessed: 2025-01-22.
22. Alec Muffett. Facebook password hashing & authentication. <https://www.youtube.com/watch?v=7dPRFoKteIU>. [Online; accessed 21-February-2023].
23. Colin Percival. Stronger key derivation via sequential memory-hard functions. In *BSDCan 2009*, 2009.
24. Jonas Schneider, Nils Fleischhacker, Dominique Schröder, and Michael Backes. Efficient cryptographic password hardening services from partially oblivious commitments. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016*, pages 1192–1203. ACM Press, October 2016.
25. Virgil Security. Virgil phe go. <https://github.com/VirgilSecurity/virgil-phe-go>, 2025. Commit 7acc0de, available at <https://github.com/VirgilSecurity/virgil-phe-go/tree/7acc0de>.
26. Security Standards Council. Pci dss v4.0.1 (section 15, 3.7.4 & 3.7.5). [https://docs-prv.pcisecuritystandards.org/PCI%20DSS/Standard/PCI-DSS-v4\\_0\\_1.pdf](https://docs-prv.pcisecuritystandards.org/PCI%20DSS/Standard/PCI-DSS-v4_0_1.pdf). [Online; accessed 21-January-2025].

## A Full Security Proofs of HildeGUARD

In this section, we recite OM-gapDH and DDH and formally prove that HildeGUARD is a hiding and binding PHE scheme.

**Theorem 1.** *Let  $\Pi_{\text{Enc}}$  be a CCA-secure private key encryption scheme,  $\Pi_{\text{NIZK}}$  be a complete, sound, and zero-knowledge NIZK proof system, and OM-gapDH and DDH be hard in  $\mathbb{G}$ . If the hash functions  $H_S, H_R, H_B, H_F$ , and  $H_N$  are modeled as random oracles, then HildeGUARD achieves hiding and binding.*

### A.1 Hardness Assumptions

We define the assumptions that we base the security of HildeGUARD on in the following.

*Decisional Diffie-Hellman Assumption.* We define the DDH assumption as a slight variation with two instances.

**Definition 4 (DDH).** *The Diffie-Hellman assumption holds in a group  $\mathbb{G}$  of prime order  $q$  if for any PPT adversary  $\mathcal{A}$  there exists a negligible function  $\text{negl}(\cdot)$  such that*

$$|\Pr[\text{DDH}_{\mathcal{A}}^0(\mathbb{G}) = 1] - \Pr[\text{DDH}_{\mathcal{A}}^1(\mathbb{G}) = 1]| \leq \text{negl}(\lambda),$$

where the randomness is taken over the random coins of the experiment and the adversary.

| DDH <sub><math>\mathcal{A}</math></sub> <sup><math>b</math></sup> ( $\mathbb{G}$ )                                                                                                                                                                                                                                            |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $x, y_0, y_1, z_0, z_1 \leftarrow \mathbb{Z}_q^*$<br>$X \leftarrow g^x, Y_0 \leftarrow g^{y_0}, Y_1 \leftarrow g^{y_1}$<br><b>if</b> $b$ <b>then</b> $Z_0 \leftarrow g^{xy_0}, Z_1 \leftarrow g^{xy_1}$<br><b>else</b> $Z_0 \leftarrow g^{z_0}, Z_1 \leftarrow g^{z_1}$<br><b>return</b> $\mathcal{A}(X, Y_0, Y_1, Z_0, Z_1)$ |

**Fig. 12.** Security game for DDH.

*One-More Gap Diffie-Hellman Assumption.*

**Definition 5 (OM-gapDH).** *The One-More Gap Diffie-Hellman assumption holds in a group  $\mathbb{G}$  of prime order  $q$  if for any PPT adversary  $\mathcal{A}$  there exists a negligible function  $\text{negl}(\cdot)$  such that*

$$\Pr[\text{OM-gapDH}_{\mathcal{A}}(\mathbb{G})] \leq \text{negl}(\lambda),$$

where the randomness is taken over the random coins of the experiment and the adversary.

### A.2 Hiding

**Lemma 1 (Hiding).** *Let  $\Pi_{\text{Enc}}$  be a CCA-secure private key encryption scheme,  $\Pi_{\text{NIZK}}$  be a complete, sound, and zero-knowledge NIZK proof system, and OM-gapDH and DDH be hard in  $\mathbb{G}$ . If the hash functions  $H_S, H_R, H_B, H_F$ , and  $H_N$  are modeled as random oracles, then HildeGUARD achieves hiding.*

*Proof (Hiding).* The idea of our proof is to incrementally update the challenge ciphertext, through various proof hops, so that it eventually consists only of random elements. Once the ciphertext is independent of the password, the only way to obtain information is by querying the oracles, and the adversary's probability of success reduces to the guessing probability. In the following, we prove hiding through a series of hybrids, where each hybrid is slightly different from the previous one, and we show that these changes are noticeable by the adversary with at most negligible probability.

| OM-gapDH <sub>A</sub> (G)                                                                | Targ()                                  | Help(z)              |
|------------------------------------------------------------------------------------------|-----------------------------------------|----------------------|
| $k \leftarrow \mathbb{Z}_q^*$                                                            | $c_t \leftarrow c_t + 1$                | $c \leftarrow c + 1$ |
| $c, c_t \leftarrow 0$                                                                    | $x_{c_t} \leftarrow \mathbb{G}$         | <b>return</b> $z^k$  |
| $\mathbb{O} := \{\text{Targ}, \text{Help}, \text{DDH}\}$                                 | <b>return</b> $x_{c_t}$                 |                      |
| $(i_1, \sigma_1), \dots, (i_\ell, \sigma_\ell) \leftarrow \mathcal{A}^{\mathbb{O}}(g^k)$ | $\text{DDH}(x_1, y_1, x_2, y_2)$        |                      |
| <b>assert</b> $c < \ell$                                                                 | $d_1 \leftarrow \text{dlog}_{x_1}(y_1)$ |                      |
| <b>assert</b> $\forall \alpha : i_\alpha \leq c_t$                                       | $d_2 \leftarrow \text{dlog}_{x_2}(y_2)$ |                      |
| <b>assert</b> $\forall \alpha \neq \beta : i_\alpha \neq i_\beta$                        | <b>return</b> $d_1 = d_2$               |                      |
| <b>return</b> $\forall \alpha : x_{i_\alpha}^k = \sigma_\alpha$                          |                                         |                      |

**Fig. 13.** Security game for OM-gapDH.

$\text{Exp}_0(\lambda)$ . The initial experiment is identical to the hiding experiment. Hence, it holds that

$$\Pr[\text{Hid}_{\text{PHE}, \mathcal{A}, Q_{\text{dec}}, \mathcal{PW}}(\lambda) = 1] = \Pr[\text{Exp}_0(\lambda) = 1].$$

$\text{Exp}_1(\lambda)$ . In the first experiment, we simulate the random oracles  $\text{H}_R$  as follows. At first, we create a table and respond to unseen queries by sampling a random exponent  $a \leftarrow \mathbb{Z}_q^*$  and return  $g^a$ . We store the query input together with  $a$  and  $g^a$  in the table. For already-seen queries, we respond with the corresponding  $g^a$  stored in the table to ensure consistency throughout different invocations. This experiment is functionally equivalent to the previous one and therefore

$$\Pr[\text{Exp}_0(\lambda) = 1] = \Pr[\text{Exp}_1(\lambda) = 1].$$

$\text{Exp}_2(\lambda)$ . In this experiment, we abort upon collisions within the random oracles  $\text{H}_S, \text{H}_R, \text{H}_F$ , and  $\text{H}_N$ . For each random oracle, the probability of a collision occurring can be upper bounded by the birthday paradox, resulting in  $\Pr[\text{coll}_{\text{H}_x}] \leq \frac{q_{\text{H}_x}}{|\mathbb{G}|}$  with  $q_{\text{H}_x}$  being the number of queries to  $\text{H}_x$ . Because the adversary is PPT, all  $q_{\text{H}_x}$  are at most polynomial in the security parameter, resulting in a negligible probability for an abortion.  $\text{Exp}_2$  is only distinguishable from  $\text{Exp}_1$ , if it aborts and therefore

$$|\Pr[\text{Exp}_1(\lambda) = 1] - \Pr[\text{Exp}_2(\lambda) = 1]| \leq \text{negl}(\lambda).$$

$\text{Exp}_3(\lambda)$ . This experiment is functionally equivalent to  $\text{Exp}_2(\lambda)$  with the difference that we abort if the first invocation of the random oracle  $\text{H}_R$  on the challenge nonce  $n^*$  is not during the challenge encryption. Since  $n_S^*$  is sampled uniformly at random and  $\text{H}_N$  has no collisions, this only happens if, by chance,  $n^*$  is equivalent to any input given in previous oracle calls. The probability of this happening is  $\frac{q_{\text{H}_R}}{2^\lambda}$  with  $q_{\text{H}_R}$  being the number of queries to  $\text{H}_R$ . Because the adversary is PPT,  $q_{\text{H}_R}$  is polynomially bound, resulting in a negligible probability for an abortion.  $\text{Exp}_3$  is only distinguishable from  $\text{Exp}_2$ , if it aborts and therefore

$$|\Pr[\text{Exp}_2(\lambda) = 1] - \Pr[\text{Exp}_3(\lambda) = 1]| \leq \text{negl}(\lambda).$$

$\text{Exp}_4(\lambda)$ . In this experiment, we make use of the OM-gapDH and DDH assumption defined in Definitions 4 and 5. During the challenge encryption, we defer from our usual way of answering queries to  $\text{H}_R$  on inputs  $(n^*, 0)$  and  $(n^*, 1)$ . Instead of sampling fresh exponents  $a_b$  (for  $b \in \{0, 1\}$ ), we inject the DDH challenge  $(X, Y_0, Y_1, Z_0, Z_1)$  by responding with  $Y_b$  to  $(n^*, b)$ . We store  $Y_b$  in the table with  $(n^*, b)$  but without an exponent  $a'_b$  such that  $Y_b = g^{a'_b}$ .

Furthermore, we answer all queries to  $\text{H}_B$  with challenge elements from **Targ**. This experiment is functionally equivalent to the previous one and therefore

$$\Pr[\text{Exp}_3(\lambda) = 1] = \Pr[\text{Exp}_4(\lambda) = 1].$$

We now build hybrids over each invocation of an honest key rotation, where we gradually remove the information stored in the challenge ciphertext.



$\text{Exp}_{4,0}(\lambda)$ . This experiment is equivalent to  $\text{Exp}_4(\lambda)$ .

$\text{Exp}_{4,q_{\text{rot}}}(\lambda)$ . In this experiment, the challenge ciphertext  $C^*$  is independent from the challenge password  $\text{pwd}^*$  and the challenge key  $K^*$  during all epochs.

$\text{Exp}_{4,i}(\lambda)$ . This experiment differs from the previous one ( $\text{Exp}_{4,i-1}(\lambda)$ ) in the epoche after the  $i$ -th key-rotation. We distinguish between two cases depending on the corruption during this epoch. We assume, without loss of generality, that the adversary corrupts either the server or the ratelimiter.

*Case 1: Corruption of S.*

$\text{Exp}_{4,i,0}^S(\lambda)$ . This experiment is equivalent to  $\text{Exp}_{4,i}(\lambda)$ .

$\text{Exp}_{4,i,1}^S(\lambda)$ . In this experiment, we compute the NIZK proofs sent by the ratelimiter using a simulator that can generate proofs for a statement without knowing the corresponding witness. Such a simulator is guaranteed to exist due to the zero-knowledge property of the proof system  $\Pi_{\text{NIZK}}$ . Because  $\text{Exp}_{4,i,1}^S(\lambda)$  only differs from  $\text{Exp}_{4,i,0}^S(\lambda)$  in the simulated proofs, every adversary that distinguishes  $\text{Exp}_{4,i,1}^S(\lambda)$  from  $\text{Exp}_{4,i,0}^S(\lambda)$  also distinguishes the simulated proofs from the real proofs. Therefore, the adversary also breaks the zero-knowledge property of the proof system. Consequently, it holds that

$$\left| \Pr \left[ \text{Exp}_{4,i,0}^S(\lambda) = 1 \right] - \Pr \left[ \text{Exp}_{4,i,1}^S(\lambda) = 1 \right] \right| \leq \text{negl}(\lambda).$$

$\text{Exp}_{4,i,2}^S(\lambda)$ . In this experiment, we replace the hashes of the challenge nonce raised to the ratelimiter secret key  $H_R(n^*, b)^{sk_R}$  with a uniformly random value  $Z_b$ . Instead of sampling a fresh ratelimiter key, we use the first component of the DDH-tuple  $(X, Y_0, Y_1, Z_0, Z_1)$  from  $\text{Exp}_4$  as the ratelimiter's public key  $pk_R = X$ . Note that this is indistinguishable in the adversary's view because it never sees  $\tau$  and there is always a matching  $\alpha$  for every pair of old and new ratelimiter key  $\alpha \leftarrow sk_R' - sk_R$ . During encryption and decryption protocols, we compute  $H_R(n, b)^{sk_R} = (g^a)^{sk_R}$  as  $X^a$ , with  $a$  taken from the table introduced in  $\text{Exp}_1$ . For the challenge nonce  $n^*$ , we compute  $H_R(n^*, b)^{sk_R} = Y_b^{sk_R}$  as  $Z_b$ . When the challenge ciphertext  $C^*$  is updated during the update protocol, we extract  $H_R(n^*, 0)^{sk_R}$  from  $t_0$  and replace it with  $Z_0$ . In the same way, we extract  $H_R(n^*, 1)^{sk_R}$  from  $t_1$  and replace it with  $Z_1$ . If there exists an adversary that distinguishes  $\text{Exp}_{4,i,2}^S$  from  $\text{Exp}_{4,i,1}^S$ , then we can use this adversary to break DDH. Consequently, it holds that

$$\left| \Pr \left[ \text{Exp}_{4,i,1}^S(\lambda) = 1 \right] - \Pr \left[ \text{Exp}_{4,i,2}^S(\lambda) = 1 \right] \right| \leq \text{negl}(\lambda).$$

It is important to see that at this point, the only things that the adversary has are  $t_0 = h_S \cdot Z_0$  and  $t_1 = Z_1 \cdot K^*$  where all values but  $h_S$  are uniformly random. His best option is to guess  $\text{pwd}^*$ , which is the only value that was sampled from a sub-exponential set. Per password guess  $\text{pwd}$ , he obtains a potential  $\overline{h_{R,0}} \leftarrow t_0 / H_S(\text{pwd}, n^*)$ . Because the correct  $h_{R,0} = H_R(n^*, 0)^{sk_R} = Z_0$  is uniformly random, the adversary can only tell if a guess was correct by interacting with the honest ratelimiter in the decryption protocol modeled by the decryption oracle. We show that the adversary can only test one  $\overline{h_{R,0}}$  per interaction with the honest ratelimiter. By sending  $x \leftarrow H_B(\overline{h_{R,0}})^{rs}$  to the ratelimiter, the adversary learns whether  $\overline{h_{R,0}} = Z_0$  because of the following equation:

$$\begin{aligned} h'_{R,1} &\leftarrow y_1 / H_F(y_0^{rs^{-1}}) \\ &= H_F(H_B(Z_0)^{rR}) \cdot h_{R,1} / H_F(H_B(\overline{h_{R,0}})^{rR}). \end{aligned}$$

If  $\overline{h_{R,0}} = Z_0$ , the  $h'_{R,1} = h_{R,1}$  and the NIZK  $\pi$  verifies. In the next and last hybrid experiment, we show that the adversary only learns this equality for exactly the  $\overline{h_{R,0}}$  that he used in the computation of  $x$ . If he can somehow compute  $H_F(H_B(\widetilde{h_{R,0}})^{rR})$  for  $\widetilde{h_{R,0}} \neq \overline{h_{R,0}}$ , he breaks the OM-gapDH assumption.

$\text{Exp}_{4,i,3}^S(\lambda)$ . In this experiment, only the decryption oracle involving  $n^*$  is changed. Upon receiving  $(x, n^*)$  from the adversary, the ratelimiter samples a random  $a \leftarrow \mathbb{Z}_q^*$  and computes  $y_0 \leftarrow \text{Help}(x)^a$ . This implicitly sets  $r_R \leftarrow k \cdot a$  with  $b$  being the internal secret of the OM-gapDH assumption. The ratelimiter samples  $y_1$  uniformly at random. To ensure that this is indistinguishable from  $\text{Exp}_{4,i,3}^S(\lambda)$ , we have to program  $H_F(h_B^{r_R})$  to be  $y_1/Z_1$ . We recognize such a query by recording all  $a$  that we used to implicitly compute  $r_R \leftarrow k \cdot a$  and checking if for the entered query  $B$ , it holds that  $B^{a^{-1}} = H_B(\cdot)^k$  for all recorded  $a$  and for all values  $H_B(\cdot)$  that the random oracle returned so far. This check can be done with the DDH oracle of the OM-gapDH assumption. Because  $H_B$  only returns challenge elements from  $\text{Targ}$ , every  $B^{1/a}$  is a solution if the above equation holds. If the adversary can compute  $H_F(H_B(\widetilde{h_{R,0}})^{r_R})$  for  $\widetilde{h_{R,0}}$  that he did not use in a previous decryption oracle query, we get an additional solution, abort and break the OM-gapDH assumption.  $\text{Exp}_{4,i,3}^S$  is only distinguishable from  $\text{Exp}_{4,i,2}^S$  if the game aborts which happens only if we break OM-gapDH and, therefore, we get

$$\left| \Pr[\text{Exp}_{4,i,2}^S(\lambda) = 1] - \Pr[\text{Exp}_{4,i,3}^S(\lambda) = 1] \right| \leq \text{negl}(\lambda).$$

Because the decryption oracle can only be queried  $Q_{\text{limit}}$  times for  $n^*$  until the ratelimiter refuses to answer (with  $Q_{\text{limit}}$  being the rate-limit implicitly used in the **ratelimit** function), the adversary can test at most  $Q_{\text{limit}}$  password guesses using the decryption oracle and we get

$$\Pr[\text{Exp}_{4,i,3}^S(\lambda) = 1] \leq \frac{Q_{\text{limit}}}{|\mathcal{PW}|} + \text{negl}(\lambda).$$

*Case 2: Corruption of R.*

$\text{Exp}_{4,i,0}^R(\lambda)$ . This experiment is equivalent to  $\text{Exp}_{4,i}$ .

$\text{Exp}_{4,i,1}^R(\lambda)$ . In this experiment, we do not sample a new server secret key  $sk_S$  during the key rotation. Instead, we forward every en-/decryption with  $sk_S$  to the corresponding oracles of the CCA-game. During the update of a ciphertext that, once decrypted with the old server key  $\bar{sk}_S$ , reveals  $n^*$ , we replace  $t_0$  and  $t_1$  with uniformly random group elements. During the decryption protocol, we still use the original  $t_0$  and  $t_1$  instead of the uniformly random ones encrypted in the enrollment record to ensure correctness. If there exists an adversary that distinguishes  $\text{Exp}_{4,i,1}^R$  from  $\text{Exp}_{4,i,0}^R$ , then we can use this adversary to break the CCA-security of the encryption scheme. Consequently, it holds that

$$\left| \Pr[\text{Exp}_{4,i,0}^R(\lambda) = 1] - \Pr[\text{Exp}_{4,i,1}^R(\lambda) = 1] \right| \leq \text{negl}(\lambda).$$

Since  $t_0$  and  $t_1$  are uniformly random, the adversary learns nothing from  $C^*$ . In the decryption oracle, all the ratelimiter learns from the server is a uniformly random value  $x$  and hence learns nothing from  $x$ . The only source left is the output of the server. The server only outputs  $K^*$  if

$$y_1 / H_F(y_0^{1/r_S}) = h_{R,1}$$

holds as otherwise, because of the soundness property of  $\Pi_{\text{NIZK}}$ , the proof  $\pi$  would not verify and the server aborts. The adversary knows  $h_{R,1}$  and has free choice over  $y_0$  and  $y_1$ . The probability of guessing  $r_S$  correctly to compute  $y_1$  as  $y_1 \leftarrow h_{R,1} \cdot H_F(y_0^{r_S^{-1}})$  is negligible. Therefore, the only option left is to follow the protocol and enter the correct password to the oracle. He succeeds with probability

$$\Pr[\text{Exp}_{4,i,1}^R(\lambda) = 1] \leq \frac{Q_{\text{limit}}}{|\mathcal{PW}|} + \text{negl}(\lambda).$$

*Cases 1 & 2 combined.* Because the limits of the **ratelimit** functions are synched during each key rotation, the adversary can guess the password  $Q_{\text{limit}}$ -times in total. Hence, we get

$$\Pr[\text{Exp}_{4,i}(\lambda) = 1] \leq \frac{Q_{\text{limit}}}{|\mathcal{PW}|} + \text{negl}(\lambda)$$

and therefore, we conclude the proof.

### A.3 Binding

**Lemma 2 (Binding).** *If  $\Pi_{\text{NIZK}}$  is a sound NIZK proof system, then HildeGUARD achieves strong soundness.*

*Proof (Lemma 2).* We prove that HildeGUARD is binding by showing that an adversary  $\mathcal{A}$  that wins  $\text{Bind}_{\text{PHE}, \mathcal{A}}(\lambda)$  outputs proofs for two contradicting statements, hence breaking the soundness of  $\Pi_{\text{NIZK}}$ .

We assume towards contradiction that  $\mathcal{A}$  breaks the binding property of HildeGUARD. Because of  $b_0$ , we know that the same ciphertext  $C = C'$  and the same server secret key  $sk_5 = sk_5'$  were used by  $\mathcal{A}$  to break binding. Consequently, the decryption of the ciphertext resulted in the same  $(t_0, t_1, n)$  in both protocol executions. We know that the two protocol runs lead to two different keys  $K \neq K'$  (because of  $b_2$ ) for the same  $t_1$  such that we can extract two different

$$\begin{aligned}\overline{h_{R,1}} &\leftarrow t_1/K \\ \overline{h_{R,1}}' &\leftarrow t_1/K'.\end{aligned}$$

In none of the two protocol executions, the server aborted ( $b_1$  tells us that  $K \neq \perp$  and  $K' \neq \perp$ ), which means that the adversary provided proofs  $\pi$  and  $\pi'$  for the two statements

$$\begin{aligned}stmt &= (\text{H}_R(n, 1), \overline{h_{R,1}}, g, pk_R) \\ stmt' &= (\text{H}_R(n, 1), \overline{h_{R,1}}', g, pk_R)\end{aligned}$$

and that the server verified them successfully. From the decryption, we know that the same nonce and from  $b_0$  that the same ratelimiter public key was used, which leads to contradicting statements. This breaks the soundness of the used NIZK proof system. Because we assumed that  $\Pi_{\text{NIZK}}$  is sound, this leads to a contradiction such that  $\mathcal{A}$  that breaks binding cannot exist.